

EVIDENCE-BASED DEFECT INVESTIGATION

Defect Prevention & Defect Management — Individual Activity

Student Name: Maria Angela Dela Cruz

Section: BSIT-3A

Student No.: 2026-0001

Date Submitted: October 3, 2026

PART A — CHOOSE AND DOCUMENT ONE DEFECT (20 POINTS)

Course: Software Quality Assurance

Instructor: Prof. Alejandro R. Ortega, PhD

A1. System / application and purpose

The system is the Palengke+ price forecasting backend in backend/forecaster.py. It loads commodity price histories from SQLite, selects an ARIMA model, and computes forecast quality metrics for the app.

A2. Exact defect

The forecast metric function safe_mape used division by actual values even when the actual price was zero. This created invalid NaN or inf results when a commodity had zero or near-zero price observations, weakening the confidence of the prediction metrics.

A3. Date, place, and context

I verified this during project review on October 3, 2026 in the local Palengke+ workspace. I tested the logic in the forecasting module and validated the regression test designed for that defect.

A4. Expected vs actual result

Expected: a finite percentage error that can support reliable model evaluation. Actual: the MAPE calculation was unstable or undefined when actual prices included zero values, which made the forecast quality metric unreliable.

A5. Evidence

I attached three evidence items from the project itself: the corrected function in backend/forecaster.py, the regression test in backend/test_sqa_fixes.py, and the unittest run showing the fix is passing.

Evidence Register

Evidence 1: safe mape implemmentation

Evidence 1: Safe MAPE implementation

```
def safe_mape(actuals: pd.Series, preds: pd.Series) -> float:
    """Calculates MAPE while handling zeros and NaNs safely."""
    actuals = pd.to_numeric(actuals, errors="coerce")
    preds = pd.to_numeric(preds, errors="coerce")
    mask = ~(actuals.isna() | preds.isna() | (actuals == 0))
    if not mask.any():
        return 0.0
    errors = (actuals[mask] - preds[mask]).abs() / actuals[mask].abs()
    return float((errors * 100).mean())
```

Evidence 2: rearegression test

Evidence 2: Regression test

```
def test_safe_mape_avoids_divide_by_zero_and_nan(self):
    actuals = pd.Series([0.0, 1.0, 0.0, 2.0])
    preds = pd.Series([0.0, 1.0, 2.0, 2.0])
    mape = forecaster.safe_mape(actuals, preds)
    self.assertTrue(math.isfinite(mape))
    self.assertGreaterEqual(mape, 0.0)
```

Evidence 3: unittest output

Evidence 3: Unit test result

```
Ran 6 tests in 0.094s
OK
This confirms the defect fix and regression coverage for the forecasting metric.
```

PART B — DEFECT CLASSIFICATION & MANAGEMENT (20 POINTS)

B1. Error → Defect → Failure chain

The human error was the incorrect formula design in the MAPE logic: it divided by actual values without excluding zeros. The software defect was the missing zero-value guard in `safe_mape`. The observable failure was invalid forecast quality metrics, which could produce NaN or inf values and confuse users and analysts.

B2. Injection and discovery stage

The defect was likely injected during coding and metric implementation in the forecasting module. It was discovered during regression testing when the project added a zero-value edge-case test to verify the metric does not break. The evidence is the test case in `backend/test_sqa_fixes.py` and the implementation in `backend/forecaster.py`.

B3. Severity

High. This affects the core quality metric used to judge forecasting reliability, which can mislead the team about model performance and reduce trust in data-driven decisions.

B4. Priority

High. Even though it may not crash the entire app, it undermines the credibility of the forecasting feature and should be resolved before release or production use because it touches a central product decision metric.

B5. Triage decision

The defect should be fixed immediately as a backend forecasting issue. The likely owner is the backend/data-science team responsible for `forecaster.py` and forecast validation.

PART C — ROOT CAUSE ANALYSIS (25 POINTS)

Why 1? Because the MAPE calculation used zero values in the denominator.

Why 2? Because the `safe_mape` function did not filter out actual prices equal to zero before dividing.

Why 3? Because the developers focused on the main formula but did not handle edge-case price values common in low-price or zero-price commodities.

Why 4? Because there was no regression test specifically checking zero-value data in the forecasting metric.

Why 5? Because the project lacked a defensive validation step for boundary cases in the model evaluation pipeline.

Root cause statement: The forecast metric used an unsafe MAPE formula that divided by zero values and lacked regression protection for edge-case prices, so the metric could return invalid values and mislead model evaluation.

PART D — CONTAINMENT → CORRECTION → PREVENTION (20 POINTS)

D1. Containment

I would prevent the app from displaying invalid MAPE values by checking the quality metric before presenting it. If the valid observation count is zero or the actual value is zero, the system should show “insufficient valid data” instead of a numeric forecast quality value.

D2. Correction

The technical correction is to update the `safe_mape` function to convert values to numeric, drop NaN and zero actual entries, and return 0.0 when there are no valid observations. This ensures the metric remains finite and interpretable.

D3. Prevention

The systemic improvement is to add regression tests for zero-price cases, require code review for boundary-value logic in calculations, and include validation checks when new metrics are introduced. This protects the forecast module from future edge-case failures and increases confidence in the quality pipeline.

PART E — QUALITY METRIC & PERSONAL REFLECTION (15 POINTS)

E1. Metric selected

Severity distribution

E2. Compute/construct the metric

This case has one defect classified as High severity out of one total defect. Severity Distribution = $(1 / 1) \times 100\% = 100\%$ High severity.

E3. Interpretation

This metric indicates that the identified issue is concentrated in one critical quality area of the forecast module. It tells us the project risk is concentrated in one model-evaluation problem, but it does not tell us how many hidden defects remain in the rest of the system or how often they occur.

E4. Personal reflection

I learned that even a small formula bug can create serious data-quality problems in a forecasting system. A metric may look professional, but if it is not validated against edge cases, it can mislead users and stakeholders. I would be more careful in reviewing ratio formulas and boundary conditions before releasing the model. I would also insist on regression tests for zero-price and empty-data scenarios. This experience taught me that quality assurance is not only about finding crashes but also about protecting the meaning of metrics themselves.

STUDENT AUTHENTICITY DECLARATION

I certify that the defect case, observations, evidence, analysis, calculations, and reflection in this submission represent my own work. I understand that I may be asked to explain my evidence and reasoning orally, and I confirm that I did not fabricate screenshots, logs, test results, or other evidence.

Student Signature: Maria Angela Dela Cruz

Date: October 3, 2026